

TROUBLESHOOTING GUIDE

Troubleshooting Guide

Repo-wide (any cluster)

SCOPE Repo-wide (any cluster)

AUDIENCE Anyone operating an RKE2-on-Proxmox cluster from this repo

CLASSIFICATION Internal — keep current as the platform evolves

Contents

01	Cluster health — quick checks	3
02	Cluster-level incident pointers	3
03	Issues that stay here (repo-level / not per-cluster)	4
04	Useful diagnostic commands (generic)	6
05	See also	6

Scope: generic troubleshooting that applies to any RKE2-on-Proxmox cluster built from this repo. For **per-cluster incident response** with specific node IPs, services, and dashboards, see the cluster-specific runbook:

- `dealing` cluster → `runbook-dealing.md`

When in doubt during an incident on the `dealing` cluster, **open the runbook first** — it has Symptoms / Diagnosis / Common causes / Remediation / Verification steps tailored to the actual topology (VIP `10.10.120.138`, masters `.131-.133`, workers `.134-.136`, Mimir at `mimir.stackflow.org`, Grafana alerts to Microsoft Teams).

Cluster health — quick checks

```
# Run on any cluster, set KUBECONFIG first.
kubectl get nodes -o wide
kubectl get pods -A | grep -v "Running\|Completed"
kubectl top nodes
kubectl top pods -A --sort-by=memory | head -20
kubectl get --raw '/readyz?verbose' | grep -v ok$
```

For the dealing cluster, `runbook-dealing.md` has a more thorough "is anything wrong?" pass with 8 ordered checks including Hubble drop verdicts and Prometheus scrape state.

Cluster-level incident pointers

These all used to be in this file. They now live in the cluster runbook because they need cluster-specific commands and dashboards.

Symptom	See
<code>kubectl</code> slow / apiserver returning 5xx / VIP unreachable	<code>runbook-dealing.md</code> → IR-1: Apiserver slow or unreachable
etcd alert firing / <code>etcd_server_has_leader == 0</code> / quota approaching	<code>runbook-dealing.md</code> → IR-2: Etcd degraded
A node shows <code>NotReady</code>	<code>runbook-dealing.md</code> → IR-3: Node goes NotReady
Pod is <code>CrashLoopBackOff</code> / <code>OOMKilled</code> / <code>Pending</code>	<code>runbook-dealing.md</code> → IR-4: Pod CrashLooping / OOMKilled / Pending
DNS resolution failing inside pods	<code>runbook-dealing.md</code> → IR-5: DNS resolution failing

Symptom	See
Pod can't reach another pod / service / external endpoint	runbook-dealing.md → IR-6: Pod can't reach another pod / service
Certificate not renewing / TLS expired	runbook-dealing.md → IR-7: Certificate renewal failed
Prometheus stops writing to Mimir / gap in dashboards	runbook-dealing.md → IR-8: Prometheus stops writing to Mimir
kube-vip VIP not failing over	covered as part of IR-1

For a *new* cluster (not dealing), copy [runbook-dealing.md](#) as a starting template, replace the IPs and hostnames, and rename to [runbook-<cluster-name>.md](#).

Issues that stay here (repo-level / not per-cluster)

Terraform apply fails against Proxmox

These are IaC-layer problems, not cluster-runtime problems. They happen during [terraform apply](#), before you have a cluster at all.

```
# Error: "400 Bad Request" on VM creation
# Cause: vm_id already in use
qm list  # On the Proxmox host – find a free range

# Error: "connection refused" on SSH provisioner
# Cause: VM still finishing cloud-init
# Fix: raise the create timeout in main.tf:
#   timeouts { create = "30m" }
# Or just re-run terraform apply once cloud-init has settled.

# Error: "template not found"
# Cause: vm_template_id in tfvars doesn't match your Proxmox template
qm list | grep <template-id>

# Error: "permission denied" on Datastore action
# Cause: API token role missing privileges
# See `multi-cluster-runbook.md` for the full pveum role command
```

Alloy not sending logs/metrics to central Loki/Mimir

Alloy runs as **systemd on the host** (not as a Kubernetes pod), so the diagnostic flow is host-level:

```

NODE_IP=10.10.120.131  # or whichever node

ssh ubuntu@$NODE_IP "
  sudo systemctl status alloy
  sudo journalctl -u alloy -n 100 --no-pager | tail -50
  sudo grep -E 'remote_write|external_labels' /etc/alloy/config.alloy | head -20
"

# Restart Alloy
ssh ubuntu@$NODE_IP "sudo systemctl reload alloy" # SIGHUP, no metric/log gap
ssh ubuntu@$NODE_IP "sudo systemctl restart alloy" # full restart if reload didn't help

# Verify data is arriving in central Loki / Mimir
# In Grafana → Explore:
#   Loki: {cluster="dealing", job="kubernetes-pods"}
#   Mimir: up{cluster_name="dealing"}

```

If the cluster sees Alloy logs/metrics under `cluster` instead of `cluster_name` (or vice versa), see [cluster-handbook-dealing.md](#) §13 for the convention this repo uses.

Generic certificate rotation

RKE2 auto-rotates kubelet / control-plane certs before expiry (`kubelet-arg: rotate-certificates=true`).

```

# Check what certs RKE2 manages
sudo /var/lib/rancher/rke2/bin/kubectrl --kubeconfig /etc/rancher/rke2/rke2.yaml get csr

# Manual full rotation (rare – only if RKE2 auto-rotation broke)
sudo rke2 certificate rotate

# For Kubernetes user kubeconfigs generated by generate-kubeconfigs.sh:
./rbac/scripts/generate-kubeconfigs.sh # re-issues all kubeconfigs

```

For **cert-manager-issued certs** on Ingress (`argocd.dealing.internal` , etc.) — that's a cert-manager flow, not an RKE2 flow. See IR-7 in the cluster runbook.

Useful diagnostic commands (generic)

```
# Capacity overview per namespace
kubectl resource-capacity --namespace --pods # needs `kubectl-resource-capacity` plugin

# All events, newest first
kubectl get events -A --sort-by='.lastTimestamp' | tail -30

# Per-job apiserver latency snapshot
kubectl get --raw /metrics | grep apiserver_request_duration_seconds_bucket | tail -20

# Spin up a network-debug pod
kubectl run nettest --image=nicolaka/netshoot --rm -it -- /bin/bash

# Privileged node debug (mounts host /, lets you ip/iptables/journalctl)
kubectl debug node/<node-name> -it --image=nicolaka/netshoot

# Etcd compact + defrag (free disk after high churn)
ETCD_POD=$(kubectl -n kube-system get pod -l component=etcd -o name | head -1)
kubectl -n kube-system exec "$ETCD_POD" -c etcd -- /bin/sh -c '
    REV=$(etcdctl --cacert=/var/lib/rancher/rke2/server/tls/etcd/server-ca.crt \
        --cert=/var/lib/rancher/rke2/server/tls/etcd/server-client.crt \
        --key=/var/lib/rancher/rke2/server/tls/etcd/server-client.key \
        endpoint status --write-out=json | python3 -c "import sys,json;
print(json.load(sys.stdin)[0][\"Status\"][\"header\"][\"revision\"])"
    etcdctl ... compact $REV
    etcdctl ... defrag
'
```

The RKE2 etcd image is distroless — `kubectl exec` may not find `/bin/sh`. If so, ssh to a master and run `etcdctl` directly with `--endpoints=https://127.0.0.1:2379`.

See also

- [runbook-dealing.md](#) — cluster-specific incident response (8 IRs + common ops + cheat sheet)
- [monitoring-playbook-dealing.md](#) — how to read each dashboard, panel thresholds, daily checklist, RCA workflow
- [cluster-handbook-dealing.md](#) — what every component does (24 sections + port inventory)
- [workload-catalog-dealing.md](#) — what's actually running where
- [policy-reviewer.md](#) — checklist before applying a NetworkPolicy/CNP/CCNP/Kyverno policy
- [backup-and-upgrade.md](#) — backup strategy + RKE2 upgrade procedure
- [scaling-procedures.md](#) — adding/removing nodes
- [multi-cluster-runbook.md](#) — bootstrapping a new cluster from the repo
- [knowledge-base.md](#) — repo-wide architecture reference